



My Cortex

AI Knowledge Engine — System Specification

Version 1.0 · **Date** May 24, 2026

Prepared by Technijian Engineering

CONFIDENTIAL — INTERNAL TECHNICAL DOCUMENT

Contents

- 1 Executive Summary
- 2 Platform Architecture
- 3 Knowledge Ingestion
- 4 AI Agents & Model Routing
- 5 Automation & Harnesses
- 6 Skills & Hooks
- 7 Memory Architecture
- 8 Data Platform
- 9 Configuration & Secrets
- 10 Consumer Access
- 11 Operations & Spend Optimization

This document describes the **as-built** My Cortex platform — its architecture, agents, automation, memory, data platform, and the model-routing strategy that keeps quality high while holding spend low.

1 Executive Summary

My Cortex is a self-improving knowledge engine that turns Technijian's inbound research stream into a structured, queryable brain. It continuously ingests Microsoft 365 mail and curated web feeds across three AI domains — **agent orchestration**, **SEO agents**, and **tech-support agents** — scores each item for relevance, writes it into an Obsidian vault and SQL Server 2025 in a single transaction, conducts autonomous deep research, and reviews itself on a weekly cadence.

The platform is built and running today. Ingestion, embeddings, deep research, repository review, wiki lint, topic refresh, and model refresh all operate unattended through a webhook service driven by scheduled automation.

3

AI DOMAINS TRACKED

18

DATABASE TABLES

6AUTOMATED
WORKFLOWS**4**

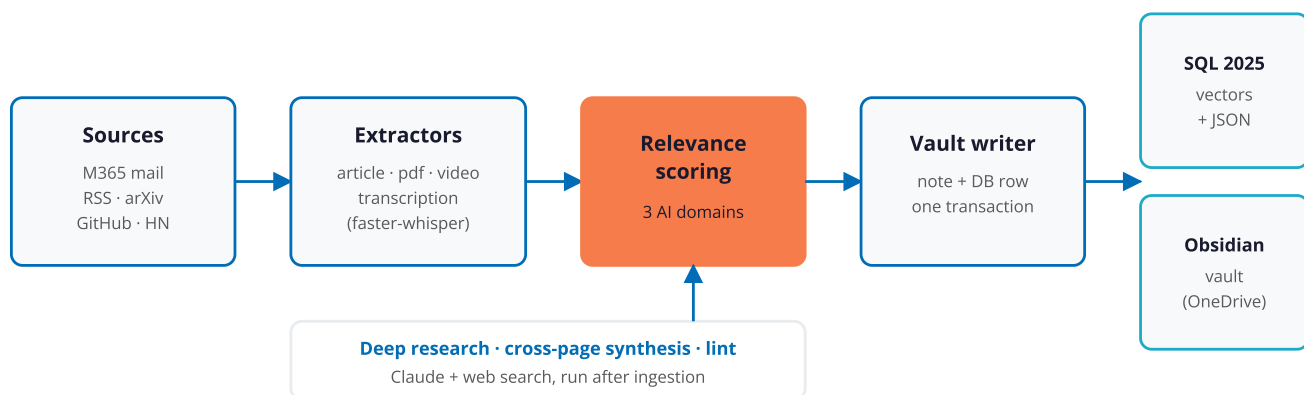
MEMORY LAYERS

A central design goal is **high-quality analysis at the lowest defensible price**. Every model call routes through one function that selects the right model per task and falls back to a known-good model on any provider error. A weekly job re-checks the model landscape and proposes cheaper or newer options for human approval.

2 Platform Architecture

My Cortex is organized as a pipeline. Sources are normalized by source-specific extractors, scored for relevance, then written once to both the vault and the database. Enrichment steps — deep research, cross-page synthesis, and lint — run after the write and feed their results back into the same stores.

Figure 1 — Knowledge flow, source to consumer



All long-running work is fronted by a FastAPI webhook service. Scheduling is handled by n8n, which calls the service on a fixed cadence; a Windows scheduled task keeps the service running across reboots. Human-facing timestamps are recorded in America/Los_Angeles.

3 Knowledge Ingestion

Mail

The mail watcher reads the `knowledge@technijian.com` shared mailbox through Microsoft Graph using certificate-based application authentication — no device login. Each processed message is moved to a **Processed** subfolder. Inbound links are cleaned of security-banner and signature noise before they become tracked sources.

Feeds

An hourly scan reviews GitHub trending across four AI categories, keeps the top results by stars, removes anything already seen, and adds genuinely new repositories. Feed definitions live in a declarative configuration file.

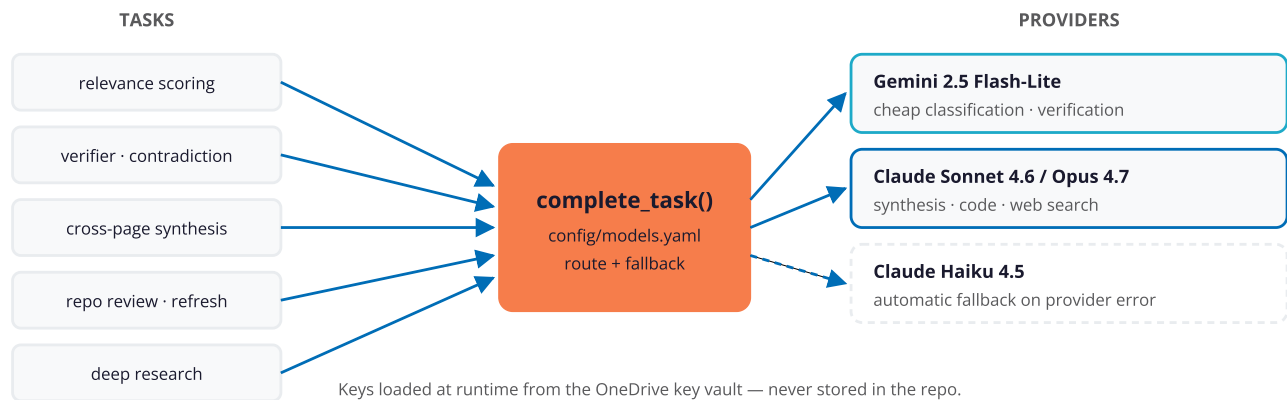
Extractors

One module handles each source type: article, PDF, arXiv, YouTube, TikTok, Twitter, Reddit, Hacker News, and GitHub. Audio and video are transcribed locally on the CPU. Every item is then scored 0–1 against the three domains; the scores decide whether the item is kept and whether deep research is triggered.

4 AI Agents & Model Routing

Every model call passes through a single routing function. A logical task name is resolved to a provider and model from one configuration file, with an automatic fallback so an unattended run survives a provider outage. Providers are either Anthropic (required for the built-in web-search tool) or any OpenAI-compatible service, reached through one shared request path.

Figure 2 — LLM task routing through `complete_task()`



Task assignments

Task	Model	Why
Relevance scoring	Gemini 2.5 Flash-Lite	High-volume 0–1 domain classification; non-reasoning, returns clean JSON, lowest price.
Deep-research verifier	Gemini 2.5 Flash-Lite	Fact-checks research articles; most source-faithful in 2026 testing.
Cross-page synthesis	Gemini 2.5 Flash-Lite	Short relation classification between related notes.
Lint contradiction	Gemini 2.5 Flash-Lite	Semantic comparison of two notes.
Repo review	Claude Sonnet 4.6	Code understanding plus nuanced improvement judgment.
Topic refresh	Claude Sonnet 4.6	Needs the Anthropic server-side web-search tool.

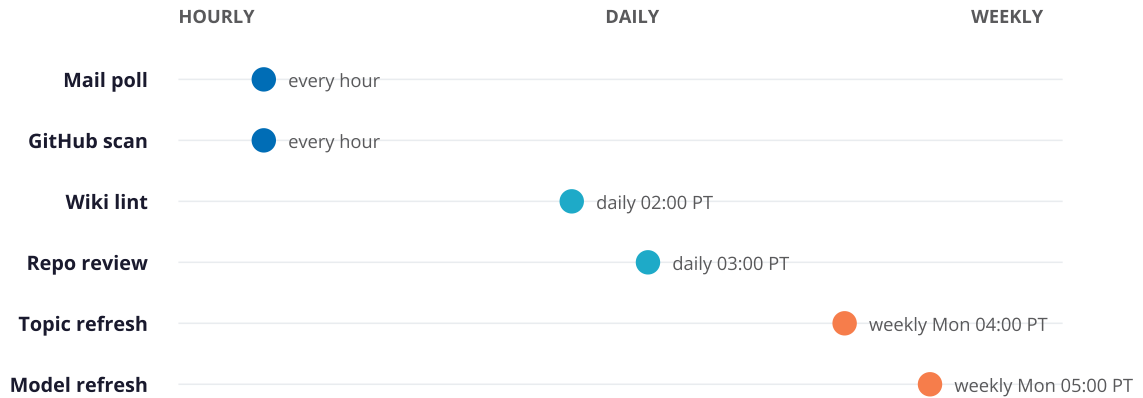
Task	Model	Why
Auto deep research	Claude Sonnet 4.6	Needs web search; cheaper than Opus with negligible quality loss on routine runs.
Manual deep research	Claude Opus 4.7	Operator-initiated, highest quality.

Short, high-volume, factual tasks run on a fast non-reasoning model for the lowest price. Quality-critical and web-search tasks run on Claude. Every cheap task falls back to Claude Haiku 4.5 if its primary provider fails.

5 Automation & Harnesses

The webhook service exposes one endpoint per workflow, protected by a shared secret. n8n triggers each endpoint on schedule and emails the operator if a run does not report success.

Figure 3 — Automated workflow cadence (n8n → webhook)



Workflow	Endpoint	Cadence
Hourly mail poll	<code>/poll</code>	every hour
Hourly GitHub scan	<code>/github-scan</code>	every hour
Daily wiki lint	<code>/lint</code>	daily 02:00 PT
Daily repo review	<code>/repo-review</code>	daily 03:00 PT
Weekly topic refresh	<code>/refresh-topics</code>	Monday 04:00 PT
Weekly model refresh	<code>/model-refresh</code>	Monday 05:00 PT

Each workflow also has a command-line equivalent for manual runs and testing.

6 Skills & Hooks

The repository ships eight Claude Code slash-command skills for vault maintenance: review, consolidate, contradictions, graduate, impact, sync, vault-status, and volatility.

Six hooks automate upkeep: vault retrieval on each prompt, topic consolidation when a session ends, a health-dashboard rebuild, code-impact checks before edits, preference extraction, and re-indexing.

7 Memory Architecture

Knowledge persists across four layers, each with a clear boundary. Durable facts live in the vault; Claude's working notes live in auto-memory; code structure is indexed separately; a consolidation pass is available but off by default.

Figure 4 — Four-layer memory architecture

Layer 1 — Obsidian vault

Durable human knowledge; topic pages, volatility tiers, git-tagged mutations.

Layer 2 — Auto-memory

Claude working notes; feedback, preferences, references. Shared across repos via OneDrive.

Layer 3 — GitNexus

Code-structure index; impact analysis before edits.

Layer 4 — Auto Dream

Consolidation pass; disabled by default.

Vault writes pass through a single writer that mirrors each note into the database in the same transaction. The vault propagates between machines through OneDrive, so the pipeline does not commit pipeline-written notes itself.

8 Data Platform

The store is SQL Server 2025 with native vector and JSON support. A versioned migration sequence defines the schema, tracked in a migrations table. There are eighteen tables and seven stored procedures.

Key capabilities

- Note and pattern embeddings are 1,536-dimension vectors, generated **server-side** through the database's built-in embedding function — never in Python.
- Vector search is wrapped in a stored procedure because the embedding call cannot be inlined inside a distance expression.
- Text columns are decoded as UTF-16 to preserve multi-byte characters such as the em-dash.
- Connection access is centralized; no module issues raw database calls of its own.

Tables span sources, notes, authors, processed mail and links, feed items, relevance scores, patterns, deep-research runs, synthesis runs, system reviews, proposed and autonomous changes, benchmark snapshots, tracked libraries, and the migration ledger.

9 Configuration & Secrets

Behavior is governed by declarative configuration: domain definitions and thresholds, tracked feeds, the reviewed-repository allowlist, refresh themes, pipeline settings, and the central model-routing file.

Secrets

No keys, tokens, passwords, or certificates are stored in the repository. Every credential is read at run time from the OneDrive key vault; the environment file holds only non-secret configuration and is excluded from version control. An explicit environment variable can override the vault for testing.

This boundary is enforced in code and documented in the project guidance so every future change keeps secrets in the vault, not the repository.

10 Consumer Access

Other Technijian repositories consume the brain through a drop-in prompt that gives any Claude Code project a daily self-improvement loop against the vault. Consumer repositories need read access to the shared vault through OneDrive — not direct database access — which keeps the integration simple and safe.

A companion document records the cross-repository access details and the daily check that surfaces newly added knowledge to each consuming project.

11 Operations & Spend Optimization

A weekly model-refresh job protects both reliability and price. First it pings every routed model to confirm it still answers — catching retired model names, authentication problems, or unfunded accounts before they affect a real run. Then it researches the current market for newer or cheaper models that match or beat each task's quality, and writes a proposal for human review.

Model routing is treated as a controlled change: the job proposes, but a person edits the routing file to accept. This keeps "use the newest, cheaper model" a deliberate decision rather than a silent change to an unattended system.

Recent outcomes from this discipline: high-volume scoring moved to a fast non-reasoning model; verification moved to the most source-faithful low-price model; and a deprecated model alias was pinned to a stable version before it could disrupt a scheduled run.

About Technijian

Founded in 2000, Technijian delivers managed IT services, cybersecurity, cloud solutions, compliance support, and AI-driven development for small and mid-sized businesses. With offices in Irvine, California and India, our dedicated pod model provides 24/7 support from a team that knows your infrastructure inside and out.

Technijian · 18 Technology Dr., Ste 141, Irvine, CA 92618 · 949.379.8499 ·
technijian.com